

DSTR Reference Manual

Common Lisp S-Expression Front End and Tooling

Sambuddha Majumder and Jayanta Majumder

May 24, 2026

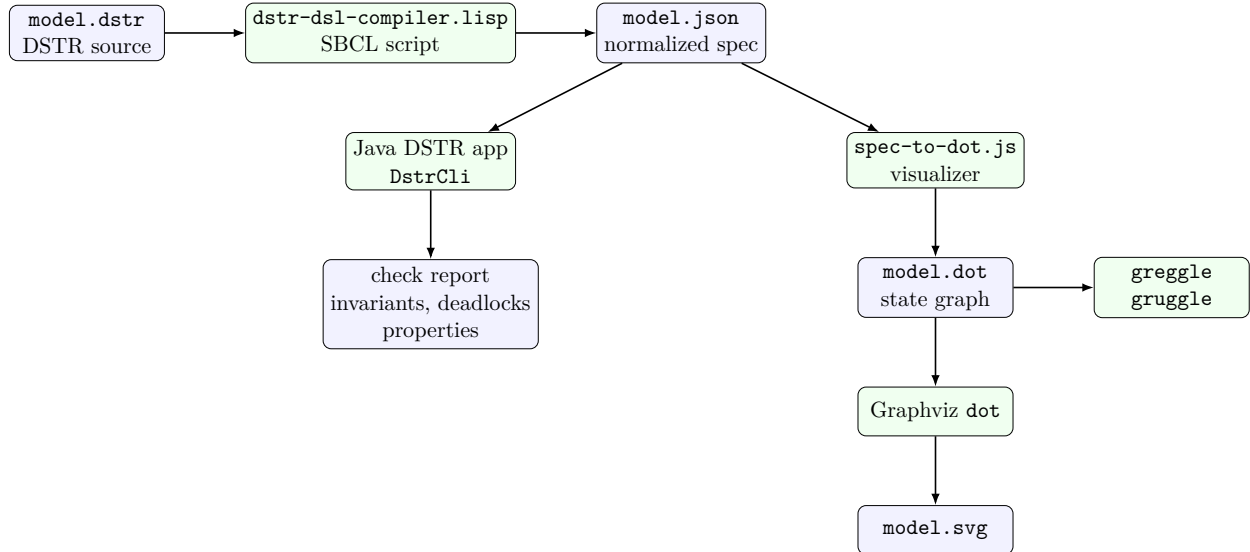
Contents

1 Purpose and Scope

DSTR is the Common Lisp s-expression front end for the **dstr** finite-state modelling prototype. A **.dstr** file describes variables, finite domains, initial states, named actions, the next-state relation, invariants, and simple reachability-style temporal properties. The DSTR compiler normalizes that source into the JSON format consumed by the existing checker and graph tooling.

The front end is intentionally close to the normalized model: current-state variables are written as ordinary names, next-state variables are written with a trailing **+**, and action references are written in the **next** clause. The Lisp reader and macro system are available before normalization, so larger models can factor repeated transition patterns using **defmacro**, **defun**, and **load**.

2 Compilation and Checking Pipeline



The Java DSTR application is the main model checker. Use it when the question is whether invariants hold, whether reachable deadlocks exist, how many transitions were explored, and whether the recorded existential properties are true. The DOT/SVG path is complementary: it gives a visual state graph for debugging, presentations, and small or bounded state spaces. The generated

.dot file is also a machine-readable graph artifact that can be fed to the `greggle` and `gruggle` graph interrogation and manipulation tools.

3 Command Reference

3.1 Compile a Single DSTR File

The direct compiler script is `scripts/dstr-dsl-compiler.lisp`. It requires SBCL.

```
sbcl --script scripts/dstr-dsl-compiler.lisp \  
    dsl-testsuite/light-switch.dstr \  
    target/light-switch.json
```

On Windows PowerShell:

```
sbcl --script scripts\dstr-dsl-compiler.lisp '  
    dsl-testsuite\light-switch.dstr '  
    target\light-switch.json
```

3.2 Compile a Directory

If the input path is a directory, every .dstr file in that directory is compiled to a corresponding .json file in the output directory.

```
sbcl --script scripts/dstr-dsl-compiler.lisp dsl-testsuite target/dstr-json
```

3.3 Run the Java Model Checker

After compiling to JSON, run the main Java application from the repository root. The Maven and Gradle application entry point is `org.dstr.cli.DstrCli`.

```
mvn exec:java -Dexec.args="target/light-switch.json"  
gradle run --args="target/light-switch.json"
```

The checker parses the normalized JSON, enumerates the finite state universe, performs reachability from `init` through `next`, checks every invariant on every reachable state, records deadlock paths, and evaluates `eventually` properties as reachability summaries. Its output includes the spec name, reachable-state count, explored-transition count, property truth values, invariant counterexample paths, and deadlock paths.

3.4 Generate DOT and SVG

The wrapper scripts compile DSTR when needed, run the Node.js graph generator, and invoke Graphviz. This is the visualization path, not a replacement for the Java model-checking CLI.

```
./dstr-to-svg.sh dsl-testsuite/light-switch.dstr  
./dstr-to-svg.sh dsl-testsuite/bakery-3proc.dstr --max-states 500
```

PowerShell:

```
.\dstr-to-svg.bat dsl-testsuite\light-switch.dstr  
.\dstr-to-svg.bat dsl-testsuite\bakery-3proc.dstr --max-states 500
```

The wrapper accepts either `input.dstr` or an already-normalized `input.json`. It writes adjacent `.json`, `.dot`, and `.svg` files. The `.svg` file is for visual inspection; the `.dot` file can also be passed to `greggle` or `gruggle` for graph interrogation and manipulation.

3.5 Graph Options

Options after the input file are passed to `scripts/spec-to-dot.js`.

- `-all-states` Include unreachable states as dashed gray nodes.
- `-rankdir TB` Draw top-to-bottom. Use LR for left-to-right.
- `-max-states N` Truncate a large explicit universe after enumerating N states.
- `-compact` Force compact node labels.
- `-no-legend` Omit the legend node in compact graphs.

4 System Form

Every source file must contain exactly one `system` form after optional top-level helper forms.

```
(system light-switch
  (vars light)
  (domain light off on)
  (init (= light off))
  (action turn-on
    (= light off)
    (= light+ on))
  (action turn-off
    (= light on)
    (= light+ off))
  (next (or $turn-on $turn-off))
  (invariant type-ok
    (in light (set off on)))
  (property eventually-on
    (eventually (= light on))))
```

4.1 Clauses

- `(vars v...)` Declares state variables in output order.
- `(vars* sep (a...) * (b...))` Declares the Cartesian product of two name groups, joining each pair with `sep`.
- `(domain v value...)` Gives `v` a finite domain. Multiple values are emitted as a set.
- `(domain* glob value...)` Applies the domain to every declared variable whose name matches the `glob` pattern.
- `(init expr...)` Defines the initial-state predicate.
- `(action name expr...)` Defines a named transition predicate.

(next expr...) Defines the next-state relation, usually as a disjunction of action references.

(invariant name expr...) Defines a predicate checked over every reachable state.

(property name expr...) Defines a property summary. Currently **eventually** is reachability-style: it is true if some reachable state satisfies the body.

When any clause body contains more than one expression, the compiler combines the body as an implicit **and**.

5 Names, Literals, and State Variables

Current variable A declared variable name such as **pc** compiles to `{"var": "pc"}`.

Next variable A declared variable with a trailing **+**, such as **pc+**, compiles to `{"next": "pc"}`.
Next variables are valid in actions and in **next**.

Action reference **\$turn-on** and the older **@turn-on** compile to `{"actionRef": "turn-on"}` inside **next**.

Symbol literal A bare undeclared symbol such as **off** compiles as the string literal `"off"`.

Number literal Numbers remain numeric.

Boolean literal **t** and **nil** compile as true and false.

Quoted literal `'foo` or `(quote foo)` forces a literal value.

6 Expression Reference

6.1 Logical Operators

```
(and (= pc idle) (= flag false))
(or $enter $exit)
(not (= pc cs))
(implies (= pc cs) (= lock held))
```

6.2 Equality, Comparison, and Arithmetic

```
(= count 0)
(!= mem 0)
(< ticket other-ticket)
(<= hr 12)
(= count+ (+ count 1))
(= remaining+ (- remaining 1))
```

The arithmetic operators **+**, **-**, *****, and **/** compile to JSON arithmetic nodes. Division is integer division in the JavaScript graph evaluator.

6.3 Sets and Membership

```
(domain pc idle trying cs)
(in pc (set idle trying cs))
(= allowed (set a b c))
```

In a `domain` clause, a sequence of values is automatically treated as a set. Use `(set ...)` when a set is needed inside an expression.

6.4 Quantifiers

```
(forall (p in (set p1 p2 p3))
  (in p (set p1 p2 p3)))

(exists (owner in (set p1 p2))
  (= owner lock-owner))
```

The quantified variable is local to the body.

6.5 Temporal Property Form

```
(property p1-can-reach-cs
  (eventually (= p1pc cs)))
```

The prototype treats this as an existential reachability question over the reachable state graph, not as a full temporal-logic liveness proof.

7 Transition Shorthands

7.1 Assignment

These three forms are equivalent in an action body:

```
(= light+ on)
(assign on to light)
(set light as on)
```

They all assert equality between the next state of `light` and the value `on`.

7.2 Equality Alias

```
(equals p_settlement_state OPEN)
```

is equivalent to:

```
(= p_settlement_state OPEN)
```

7.3 Conditional Form

```
(if
  ((equals p_settlement_state OPEN)
   (equals c1_settlement_state OPEN))
  then
  ((assign CANCELLED to p_settlement_state)
   (assign CANCELLED to c1_settlement_state)))
```

The conditional shorthand means conjunction of the condition and consequence. It is not a branching expression with an `else`; it is useful for writing guarded transition cases. A branch written as a list of sibling expressions is treated as an implicit `and`.

7.4 Frame Conditions by Glob

```
(unchanged* *_market_status *_event_type *_mission_version)
```

This expands to a conjunction of frame equalities for every declared variable matched by the glob patterns:

```
(and (= p_market_status+ p_market_status)
      (= c1_market_status+ c1_market_status)
      ...)
```

7.5 Alternate Scenarios

```
(alternate-scenarios
  (if (= state OPEN) then (assign CANCELLED to state))
  (if (= state BOOKED) then (assign state to state)))
```

`alternate-scenarios` is a readability alias for `or`.

8 Macros and Helper Functions

DSTR permits top-level `defmacro`, `defun`, and `load` forms before the `system`. Relative load paths are resolved against the current `.dstr` file's directory.

8.1 Expansion Forms

`(expand name arg...)` Macroexpand a named macro.

`(%name arg...)` Shorthand for `(expand name arg...)`.

`(!name arg...)` Expand a macro whose result is a list of sibling forms and splice those forms into the surrounding body.

8.2 Built-in Macro Helpers

```
(%same pc)           ; expands to (= pc+ pc)
(!unchanged p2pc mem) ; splices (= p2pc+ p2pc), (= mem+ mem)
```

8.3 Example Macro Library

```
(defun preserve-clauses (&rest vars)
  (mapcar (lambda (var)
            '(= ,(next-var-symbol var) ,var))
          vars))

(defmacro preserve (&rest vars)
  (preserve-clauses vars))
```

8.4 CAS Race With Spliced Actions

```
(defmacro p1-actions ()
  '((action p1-start
    (= p1pc a0)
    (= p1pc+ try)
    (!preserve p2pc mem))
    (action p1-cas-success
    (= p1pc try)
    (= mem 0)
    (= p1pc+ won)
    (!preserve p2pc)
    (= mem+ 1))))

(system cas-2proc-race
  (vars p1pc p2pc mem)
  (domain p1pc a0 try won lost)
  (domain p2pc a0 try won lost)
  (domain mem 0 1 2)
  (init (= p1pc a0) (= p2pc a0) (= mem 0))
  (!p1-actions)
  (next (or $p1-start $p1-cas-success)))
```

9 Product Variables and Glob Domains

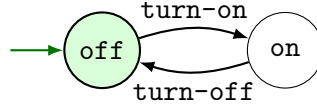
```
(system trade-cancellation
  (vars* _ (p c1 c2) * (market_status settlement_state settlement_version))
  (domain* *_market_status MATCHED)
  (domain* *_settlement_state OPEN RELEASED BOOKED CANCELLED)
  (domain* *_mission_version 1 2 3)
  (init
    (equals p_settlement_state OPEN)
    (equals c1_settlement_state OPEN)
    (equals c2_settlement_state OPEN))
  ...)
```

The `vars*` clause above declares:

`p_market_status, p_mission_state, ..., c2_mission_version.`

10 Visualization Semantics

The graph tool enumerates the finite state universe, finds states satisfying `init`, applies `next`, and annotates the reachable graph.



Green nodes are initial states, amber nodes are deadlocks, and red nodes are states where at least one invariant fails. Edge labels name the actions that were enabled for that transition.

11 Normalized JSON Shape

The compiler emits JSON in the same shape as the core checker expects:

```
{
  "name": "light-switch",
  "variables": ["light"],
  "domains": {
    "light": {
      "set": [{"lit": "off"}, {"lit": "on"}]
    }
  },
  "init": {
    "=": [{"var": "light"}, {"lit": "off"}]
  },
  "actions": [
    {
      "name": "turn-on",
      "body": {
        "and": [
          {"=": [{"var": "light"}, {"lit": "off"}]},
          {"=": [{"next": "light"}, {"lit": "on"}]}
        ]
      }
    }
  ],
  "next": {"or": [{"actionRef": "turn-on"}, {"actionRef": "turn-off"}]},
  "invariants": [],
  "properties": []
}
```

12 Practical Modelling Notes

- Every variable must have exactly one domain, either through `domain` or `domain*`.
- Actions should usually define every next-state variable, directly or via a frame condition. If a variable is not constrained, the explicit-state checker may allow many successor values.
- Use `unchanged`, `unchanged*`, or `same` to make frame conditions visible and auditable.

- Keep large product domains small while exploring a model. Use `-max-states` for graphing large universes.
- Prefer `$action` for new Lisp-style models. The older `@action` form remains accepted for compatibility.

13 Minimal Checklist

1. Declare variables with `vars` or `vars*`.
2. Provide finite domains for all variables.
3. Define an `init` predicate.
4. Define one or more `action` clauses.
5. Define `next`, normally as an `or` of action references.
6. Add invariants and reachability properties.
7. Compile to JSON, run the Java DSTR checker for the authoritative checking report, then use `spec-to-dot.js` or the SVG wrapper for graph inspection.